

SC102 L09: 程序化动画与 Tweening ——详细学习笔记

涵盖：线性与非线性动画 | AnimationCurve | Animation Event | 动画曲线表达式 | Tweening 系统

1 线性动画与非线性动画

[p.4-5]

1.1 线性动画

动画是物体属性随时间变化的行为。**线性动画**指属性变化速率恒定。

```
// 线性：匀速移动
void Update() {
    transform.Translate(Vector3.left * Time.deltaTime);
}
```

1.2 非线性动画

非线性动画中属性变化速率不是恒定的——”先快后慢”、”尾部回弹”、”蓄力冲刺”等都是非线性动画。非线性动画让游戏更生动富有表现力。

```
// 非线性示例
a = Mathf.Lerp(a, 50, 0.2f);           // 逐渐逼近
a = Mathf.Pow(a, 0.99f);               // 幂函数衰减
spriteRenderer.color = new Color(
    Mathf.Sin(Time.time) * 0.5f + 0.5f, 0, 0); // 正弦波动
```

核心洞察：基本上所有非一次函数 ($y \neq kx + b$) 都可以制作非线性动画。

2 Unity 动画编辑器中的曲线

[p.6-7]

意外发现：在 Animation 窗口做一个”0 度 → 360 度”的旋转动画，物体并不会匀速旋转，而是在某处停顿。这是因为 Unity 默认的动画曲线不是线性的。

在 Animation 窗口底部选 Curves 可查看动画曲线视图。曲线用于处理关键帧之间的平滑过渡，每个关键帧有左右两条切线定义过渡走向。要获得匀速旋转：右键关键帧 → 调整切线为 Linear。

3 AnimationCurve 类

[p.8-9]

动画曲线的定义：一个从 $[0, 1]$ 映射到 $[0, 1]$ 的函数。对于每个 0-1 之间的 x 值，采样得到对应 y 值。

Unity 的 AnimationCurve 类允许在编辑器/代码中创建并采样动画曲线：

- Evaluate(float time): 在 $x=time$ 处采样返回 y 值
- AddKey(Keyframe key): 添加关键帧
- RemoveKey(int index): 移除关键帧

注意：Unity 中曲线的 x 、 y 取值不限于 $[0, 1]$ ，但一般建议”标准化”到此范围以便 Tweening 系统使用。

4 Animation Event

[p.10]

Animation Event 允许在动画的某一帧执行一个函数。用于动画与逻辑同步（如攻击动画的特定帧播放音效）。

限制：

1. 脚本必须挂载在 Animator 所在的物体上
2. 函数只能接受 float/int/string 等基本类型，最多 1 个参数
3. 函数不能为 static；需要多参数时使用 AnimationEvent 结构体

5 常见动画曲线及其表达式

[p.13-16]

以下 x 为归一化时间 (0-1)， y 为归一化进度 (0-1)：

单端曲线 (In)：

- Linear: $y = x$

- Quadratic: $y = 1 - (1 - x)^2$ (先快后慢)
- Cubic: $y = 1 - (1 - x)^3$
- Quartic: $y = 1 - (1 - x)^4$
- Expo: $y = 2^{10(x-1)}$ (极快 $\rightarrow$ 极慢)
- Elastic: $y = -2^{-10+10x} \cdot \sin(2\pi(3.33x - 3.58))$ (弹性回弹)
- Back: $y = x^2(2.70158x - 1.70158)$ (超过目标再回来)
- Sqrt: $y = 1 - \sqrt{1 - x^2}$

双端曲线 (InOut): 分段函数, $x < 0.5$ 时使用 in 变体, $x \geq 0.5$ 时使用 out 变体 (对称翻转)。

- QuadraticDouble: $y = 2x^2$ ($x < 0.5$); $y = 2(2 - x)(x - 0.5) + 0.5$ ($x \geq 0.5$)
- CubicDouble: $y = 4x^3$ ($x < 0.5$); $y = -4(1 - x)^3 + 1$ ($x \geq 0.5$)

6 动画曲线封装类

[p.17-18]

核心设计: 用 `Func<float, float>` 表示采样函数 (输入 x 返回 y), 为每种曲线创建对应的静态方法。

```
public enum CurveType { Linear, Quadratic, Cubic,
    QuadraticDouble, CubicDouble, Elastic, Back, ... }

public class SKCurve {
    public Func<float, float> func { get; private set; }

    public static float SampleCurve(SKCurve curve, float x) {
        return curve.func(Mathf.Clamp01(x));
    }

    public static SKCurve Linear = new SKCurve(x => x);
    public static SKCurve Quadratic = new SKCurve(
        x => 1 - (1-x)*(1-x));
    // ... 更多曲线
}
```

7 Tweening 函数

[p.19-20]

Tweening = In-Betweening，处理两帧之间的平滑过渡。动画曲线是其最直观的实现方式。

目标：写一个函数，调用一次即可对物体执行一个动画效果：

```
public static Coroutine StartProcedure(
    SKCurve curve, float time, Action<float> action,
    Action onFinish = null, string id = "") {

    return CoroutineRunner.StartCoroutine(Procedure());

    IEnumerator Procedure() {
        float elapsed = 0;
        while (elapsed < time) {
            elapsed += Time.deltaTime;
            float t = Mathf.Clamp01(elapsed / time);
            float value = SKCurve.SampleCurve(curve, t);
            action(value); // 将曲线采样值传给调用方
            yield return null;
        }
        onFinish?.Invoke();
    }
}
```

使用示例：

```
// 2秒内先快后慢从point1移动到point2
SKCurve.StartProcedure(SKCurve.Quadratic, 2.0f, (t) => {
    transform.position = Vector3.Lerp(point1, point2, t);
});

// 弹性掉落效果
SKCurve.StartProcedure(SKCurve.Elastic, 1.0f, (t) => {
    transform.position = Vector3.Lerp(startPos, endPos, t);
});
```

Tweening 系统的强大之处：所有有时间跨度的效果——物件移动、颜色渐变、UI 窗口动画、文字逐字出现——都可以通过一次 `StartProcedure` 调用完成。

8 课后习题详解

[p.21]

8.1 题 1: StopProcedure 函数

```
private static Dictionary<string, Coroutine> procedures
    = new Dictionary<string, Coroutine>();

public static void StopProcedure(string id) {
    if (procedures.TryGetValue(id, out Coroutine coroutine)) {
        CoroutineRunner.StopCoroutine(coroutine);
        procedures.Remove(id);
    }
}
```

id 参数用于标识每个动画——停止时按 id 找到并终止对应协程。

8.2 题 2: 文字逐个出现（打字机效果）

```
IEnumerator TypewriterEffect(TextMeshProUGUI textUI,
    string fullText, float duration) {
    textUI.text = "";
    for (int i = 0; i < fullText.Length; i++) {
        textUI.text += fullText[i];
        yield return new WaitForSeconds(
            duration / fullText.Length);
    }
}
```

或用 Tweening: StartProcedure(curve, duration, t => textUI.maxVisibleCharacters = (int)(t * totalChars))。

8.3 题 3: DoTween 实现

DoTween 是一个流行的程序化动画插件。用 DoTween 实现 4 个需求：

```
// 1. 先快后慢移动
transform.DOMove(new Vector3(5,0,0), 2f).SetEase(Ease.OutQuad);
// 2. 先慢后快移动
transform.DOMove(new Vector3(5,0,0), 2f).SetEase(Ease.InQuad);
// 3. 颜色渐变
```

```
sprite.DOColor(Color.white, 3f).OnComplete(() =>
    sprite.DOColor(Color.black, 2f));
// 4. 弹性掉落
transform.DOMoveY(0, 1f).SetEase(Ease.OutElastic);
```

DoTween 封装了各种 Ease 曲线，一行代码完成复杂动画。